

Assignment 1 Report

Git Branching & Collaboration – MAI201 MLOps

Student: Jaskaran Singh

GitHub: gilljaskaran

Course: MAI201 MLOps

Date: June 4, 2026

Repository: <https://github.com/gilljaskaran/mlops-git-assignment-JASKARAN-SINGH>

1. GitHub Network Graph

The network graph below shows all branches created and merged during this assignment:

- feature/add-readme-details branched from develop and merged via PR #1
- feature/add-dockerignore branched from develop and merged via PR #1
- feature/add-code-of-conduct branched from develop and merged via PR #3
- feature/update-readme branched from develop, caused a merge conflict, which was resolved and merged

```
jaskaran@DESKTOP-RKDRK11 MINGW64 ~/mlops-git-assignment-JASKARAN-SINGH (feature/update-readme)
$ cd ~/mlops-git-assignment-JASKARAN-SINGH
git log --oneline --graph --all
* 56a1d56 (HEAD -> feature/update-readme, origin/feature/update-readme) Add student information
* c036775 (develop) Modify course information section
* c4a401f (origin/develop) Add course information
|
| * 8374542 Merge pull request #3 from gilljaskaran/feature/add-code-of-conduct
|
| * addbc60 (origin/feature/add-code-of-conduct, feature/add-code-of-conduct) Add contact section to code of conduct
| * 5a8cc99 Add code of conduct
|
| * 76387fb Merge pull request #2 from gilljaskaran/feature/add-dockerignore
|
| * b59e5b2 (origin/feature/add-dockerignore, feature/add-dockerignore) Add build directories to dockerignore
| * 8e88cb0 Create dockerignore file
|
| * 6d9d1aa Merge pull request #1 from gilljaskaran/feature/add-readme-details
|
| * 3394f88 (origin/feature/add-readme-details, feature/add-readme-details) Add repository structure section
| * 5c84c81 Add Project description and setup instruction
|
| * 202c9d6 (origin/main, origin/HEAD, main) Initial commit
jaskaran@DESKTOP-RKDRK11 MINGW64 ~/mlops-git-assignment-JASKARAN-SINGH (feature/update-readme)
$
```

- Require at least 1 approval: YES

1. Branch Protection Rules

Branch protection rules were configured for the **main** branch with the following settings:

- Require a pull request before merging: YES

- Dismiss stale pull request approvals when new commits are pushed: YES
- Require linear history: YES
- Allow force pushes: DISABLED
- Allow branch deletion: DISABLED

The screenshot shows the GitHub repository settings for 'mlops-git-assignment-JASKARAN-SINGH'. The 'Network' tab is selected, displaying a network graph of commits. Below the graph, the 'Branch protection rules' section is visible, showing several rules with checkboxes for various protection features.

Branch protection rules:

- ☒ **Require a pull request before merging**
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.
- ☒ **Require approvals**
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.
Required number of approvals before merging: 1
- ☒ **Dismiss stale pull request approvals when new commits are pushed**
New reviewable commits pushed to a matching branch will dismiss pull request review approvals.
- ☐ **Require review from Code Owners**
Require an approved review in pull requests including files with a designated code owner.
- ☐ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed the most recent reviewable push.
- ☐ **Require status checks to pass before merging**
Choose which [status checks](#) must pass before branches can be merged into a branch that matches this rule. A pull request must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule.
- ☐ **Require conversation resolution before merging**
When enabled, all conversations on code must be resolved before a pull request can be merged into a branch that matches this rule. [Learn more about requiring conversation completion before merging.](#)

A 'Snipping Tool' window is overlaid on the bottom right, displaying the text: 'Screenshot copied to clipboard', 'Automatically saved to screenshots folder.', and 'Mark-up and share'.

3. Git Log Output

Output of: git log --oneline --graph --all

```

* 56ald56 (feature/update-readme) Add student information
| * c036775 (develop) Modify course information section
| * c4a401f (origin/develop) Add course information
|/ * 8374542 Merge pull request #3 from gilljaskaran/feature/add-code-of-
conduct
|\ | * addbc60 Add contact section to code of
conduct | * 5a8cc99 Add code of conduct
|/ * 76387fb Merge pull request #2 from gilljaskaran/feature/add-
dockerignore
|\ | * b59e5b2 Add build directories to
dockerignore | * 8e88cb0 Create dockerignore
file
|/ * 6d9dlaa Merge pull request #1 from gilljaskaran/feature/add-readme-
details
|\
| * 3394f88 Add repository structure section | * 5c84c81
Add Project description and setup instruction
|/
* 202c9d6 Initial commit

```

4. Reflection on Merge Conflicts

The most challenging part of this assignment was resolving the merge conflict in Part 3. When both the feature/update-readme branch and the develop branch modified the README.md file, Git could not automatically decide which changes to keep and flagged the conflict with <<<<<<, =====, and >>>>>> markers.

At first, it was confusing to understand what these markers meant and which section belonged to which branch. The fix required manually editing the file in a text editor, carefully removing all three conflict markers, and making sure both the student information section (from feature/update-readme) and the course information section (from develop) were preserved in the final file.

After resolving the conflict, the file was staged and committed, and the pull request was successfully completed. This experience taught me the importance of clear communication in teams — if two developers know in advance which sections of a file they are each editing, merge conflicts can be avoided entirely.